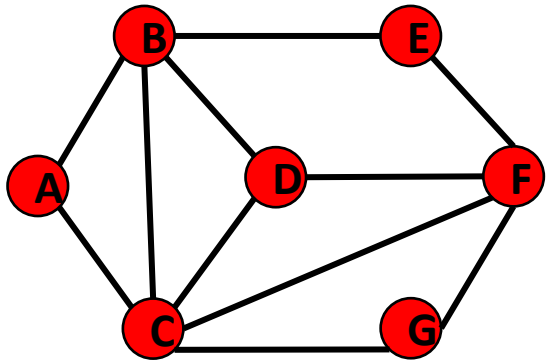
Spanning Tree

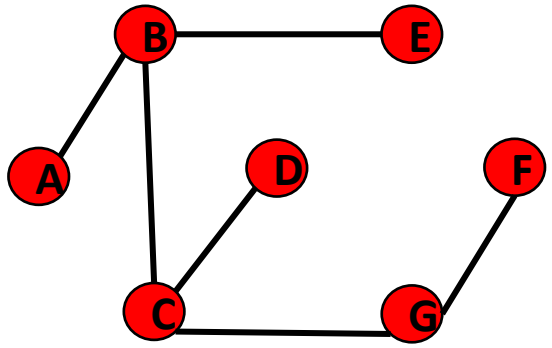
S. S. Satapathy

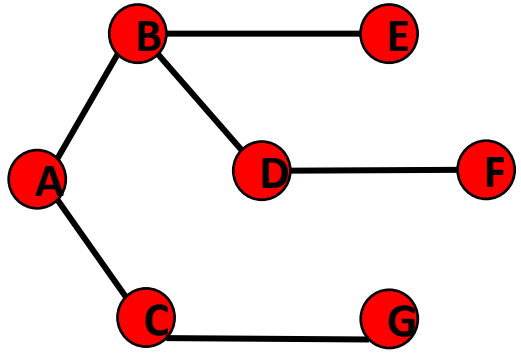
Spanning Tree

- For a given with **e edges**, we can have **2^e subgraphs** considering distinct combinations of the edges.
- Out of **these subgraphs**, there will be **some trees**. Out of these **trees**, there are **certain types of trees**, called **spanning trees** which are **defined as follows**:
  - A **tree T** is said to be a **spanning tree** of a **connected graph G if T** is a **subgraph** of **G and T contains all vertices** of **G**.
  - **Note**: Spanning trees are defined only for a **connected graph**. A **disconnected graph** with **k components** has a **spanning forest** consisting of **k spanning trees**.
- The **spanning tree** is also called the **skeleton or scaffolding** of the original graph G, **as** the **vertices** of G are barely **hanging together** in a **spanning tree**.
- Since **spanning trees** are the **largest trees** (with the **maximum number of edges**) **among** all trees in **G**, it is also called a **maximal tree subgraph** or **maximal tree** of **G**.



graph G

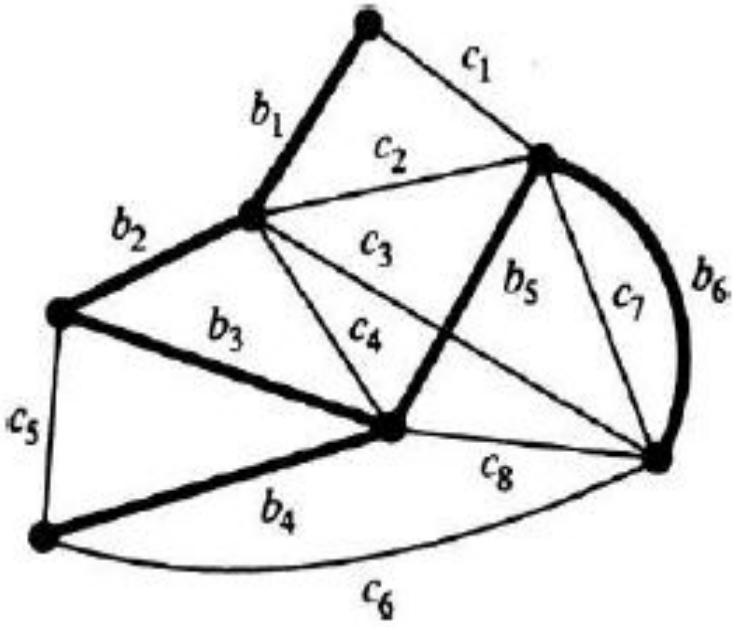




Two possible **Spanning trees** of **G**

Spanning Tree

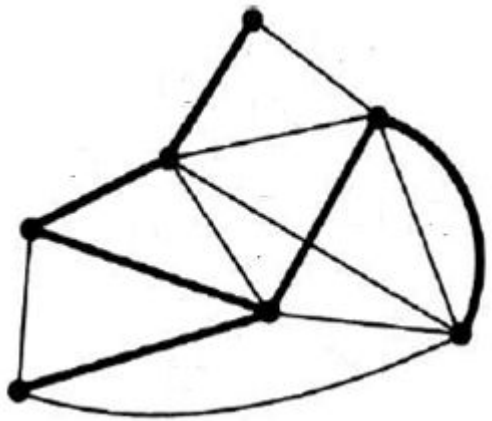
- An **edge** in a **spanning tree T** is called a **branch** of **T**. An **edge** of G that is **not in a given spanning tree T** is **called** a **chord**. For instance, **edges** $\mathbf{b_1}$, $\mathbf{b_2}$, $\mathbf{b_3}$, $\mathbf{b_4}$, $\mathbf{b_5}$, and $\mathbf{b_6}$ are **branches** of the spanning tree shown in the given figure, while edges $\mathbf{c_1}$, $\mathbf{c_2}$, $\mathbf{c_3}$, $\mathbf{c_4}$, $\mathbf{c_5}$, $\mathbf{c_6}$, $\mathbf{c_7}$, and $\mathbf{c_8}$ are **chords**.
  - **Note**: these **branches** and **chords** are **defined only with respect to** a **given spanning tree**. An **edge** that **is a branch of one spanning tree** $\mathbf{T_1}$ (in a graph G) **may be a chord** with respect to **another spanning tree** $\mathbf{T_2}$.
  - With respect to any of its **spanning trees**, a connected graph of **n vertices** and **e edges has** $\mathbf{n - 1}$ **tree branches** and $\mathbf{e - n + 1}$ **chords**.
- **Every connected graph** has **at least one spanning tree**.
- **Finding a spanning tree**:
  - If **G has no circuit**, **it is its own spanning tree**. If **G has a circuit**, **delete an edge** from the circuit. This will still leave the graph connected. **If** there are **more circuits**, **repeat** the operation till an edge from the **last circuit is deleted—leaving a connected**, **circuit-free graph** that **contains all the vertices** of **G**.



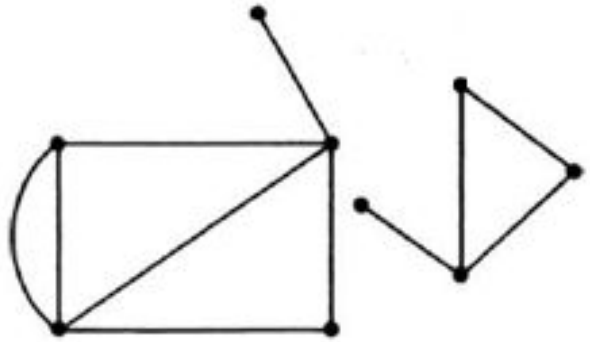
Spanning tree of a connected graph **shown in heavy lines**

Rank and Nullity in a Tree

- For any **graph G**, the **number** of **vertices n**, the number of **edges e**, and the number of **components k** are **related**.
  - Since every component of a graph must have at least one vertex, $n \geq k$.
  - The **number of edges** in a **component** can not be less than the **number of vertices** in that component **minus one**. Therefore, $e \geq n - k$.
  - Apart from the constraints $n - k \geq 0$ and $e - n + k \geq 0$, **these three numbers n**, **e**, and **k** are **independent**, and they are **fundamental numbers** in any **graph**.
- From these three numbers, two important numbers are derived called **rank** and **nullity** (also called **cyclomatic number**, or **first Betti number**), defined as
  - **rank** $r = n - k$,
  - **nullity** $\mu = e - n + k$.
- **Note**: In a connected graph, the **rank** $r = n - 1$ and **nullity** $\mu = e - n + 1$. For any graph G, **rank** of **G** = **number of branches**, **nullity** of **G** = **number of chords** **in the spanning tree (or forest)**, and **rank** + **nullity** = **number of edges** in **G**.



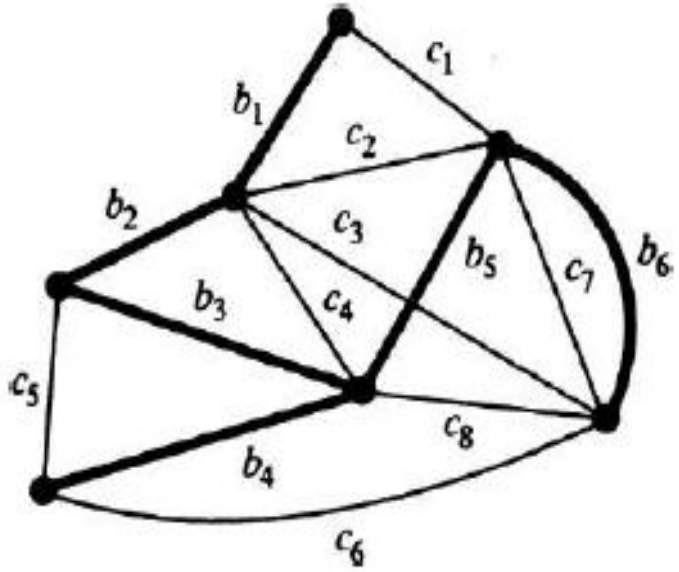
Graph G with **n=7**, **e=14**, and **k=1**. so $n - k \geq 0$ and $e - n + k \geq 0$



Graph G with **n=9**, **e=11**, and **k=2**. so $n - k \geq 0$ and $e - n + k \geq 0$

Fundamental Circuits

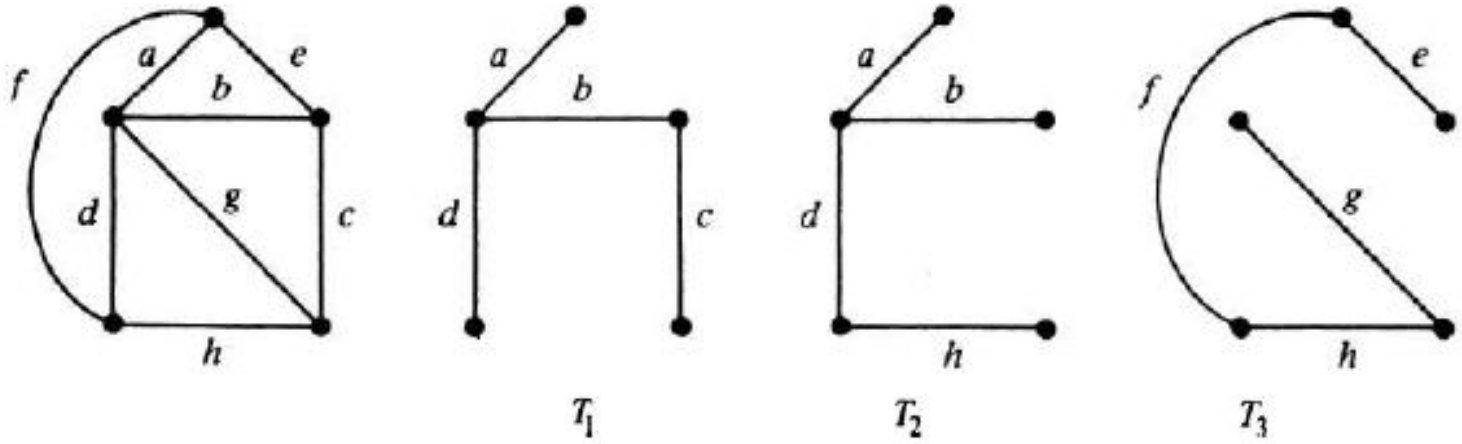
- In a **spanning tree T** of a **connected graph G**, **adding any one chord** to **T** will **create** exactly **one circuit**. Such a **circuit** is **called** a **fundamental circuit**.
- **Number** of **fundamental circuits** is exactly **equal to** the **number of chords**, **μ** (= **e** − **n** + **k**).
- **Note**:
  - a **circuit is a fundamental circuit only with respect to a given spanning tree**. A given circuit may be fundamental with respect to one spanning tree, but not with respect to a different spanning tree of the same graph. Adding c_1 to the spanning tree, we get a subgraph {b_1, b_2, b_3, b_4, b_5, b_6, c_1}, which has one circuit (fundamental circuit), {b_1, b_2, b_3, b_5, c_1}. Adding the chord c_2 (instead of c_1) to the tree, we obtained a different fundamental circuit, {b_2, b_3, b_5, c_2}. If we add both chords c_1 and c_2 to the tree, the subgraph {b_1, b_2, b_3, b_4, b_5, b_6, c_1, c_2}, has the above two fundamental circuits and also a third circuit, {b_1, c_1, c_2}, which is not a fundamental circuit.
  - Although the **number of fundamental circuits** (as well as the **total number of circuits**) **in a graph** is **fixed**, the **circuits that become fundamental** change with the **spanning trees**.



Spanning tree of a connected graph **shown in heavy lines**

Finding All Spanning Trees of a Graph

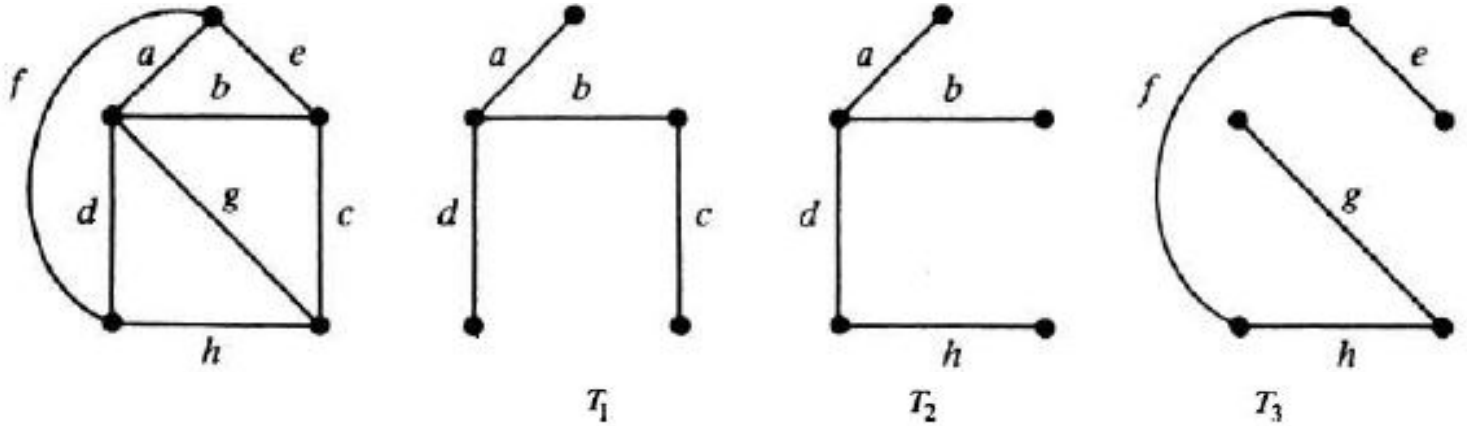
- In a given connected graph there are a large number of spanning trees.
- **Procedure for generating all spanning trees**: **start** with a **given spanning tree**, say **tree T_1** (**a b c d** in Fig). **Add** a **chord**, say **h**, to the tree T_1. This forms a **fundamental circuit** (**b c h d** in Fig). **Removal** of any branch, say **c**, **from** the fundamental circuit **b c h d** will **create** a **new spanning tree T_2**. **Instead of deleting** branch **c**, **deleting d or b** would **give two additional spanning trees a b c h** and **a c h d**.
- This **generation** of **one spanning tree from another**, **through addition of a chord** and **deletion of an appropriate branch**, is **called** a **cyclic interchange or elementary tree transformation**.



A **Graph** and **three** of its **spanning trees**

Distance Between Two Spanning Trees

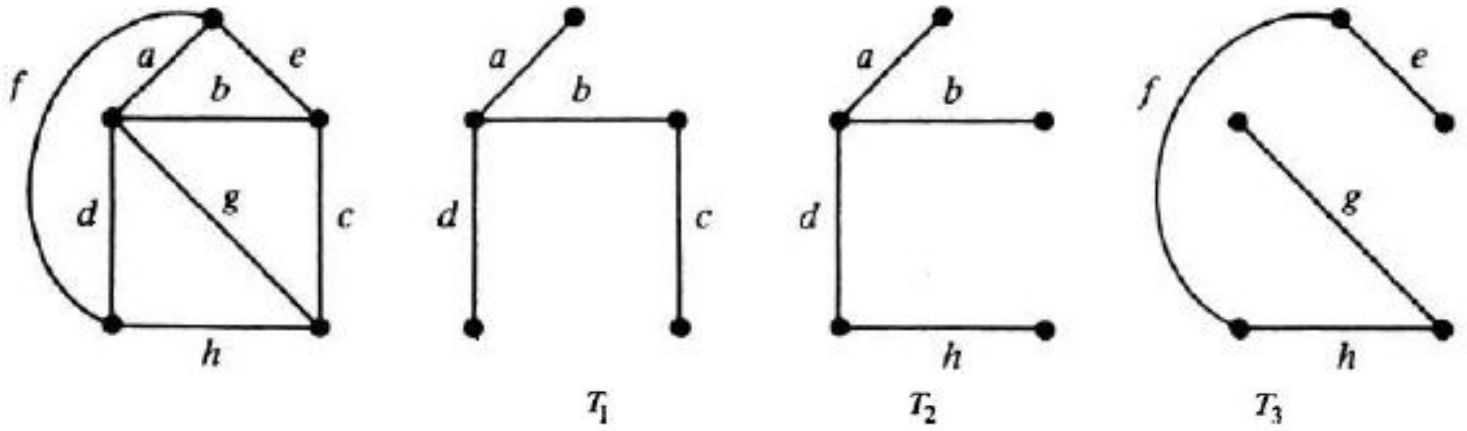
- The **distance** $d(T_i, T_j)$ **between two spanning trees** T_i and T_j of a graph G is **defined** as the **number** of **edges** of G **present in one tree but not in the other**.
  - $d(T_2, T_3) = 3$. As for T_2, set of edges = {a, b, d, h} and for T_3, set of edges = {e, f, g, h}. Edges present T_2 in but not in T_3 are {a, b, d}.
  - $d(T_3, T_2) = 3$. As for T_3, set of edges = {e, f, g, h} and for T_2, set of edges = {a, b, d, h}. Edges present T_3 in but not in T_2 are {e, f, g}.
  - $d(T_1, T_2) = 1$. As for T_1, set of edges = {a, b, c, d} and for T_2, set of edges = {a, b, d, h}. Edges present T_1 in but not in T_2 are {c}.
  - $d(T_2, T_1) = 1$. As for T_2, set of edges = {a, b, d, h} and for T_1, set of edges = {a, b, c, d}. Edges present T_2 in but not in T_1 are {h}.



A **Graph** and **three** of its **spanning trees**

Distance Between Two Spanning Trees

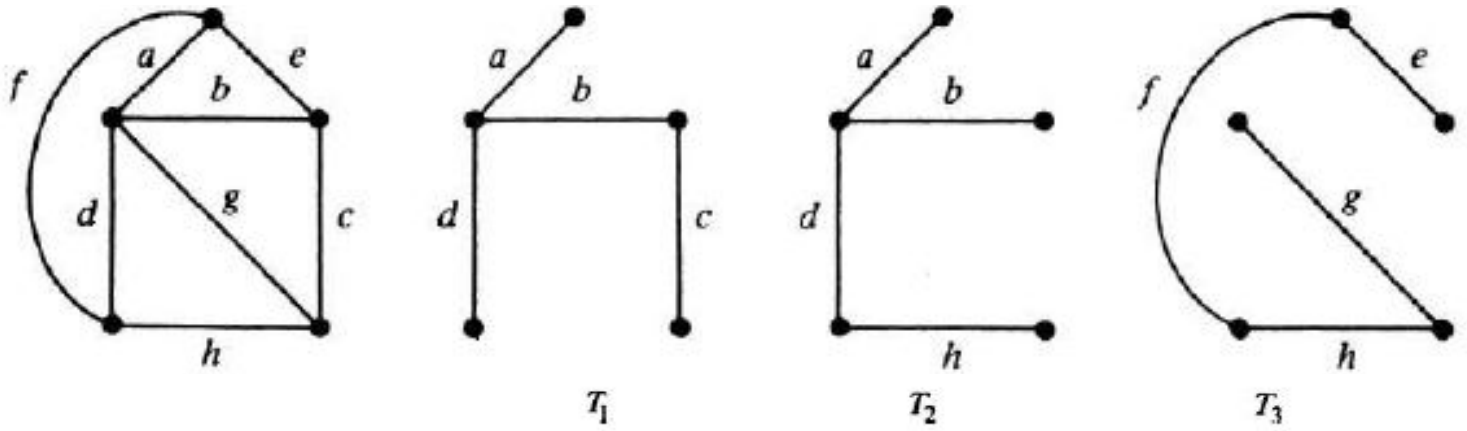
- The **distance** $d(T_i, T_j)$ **between two spanning trees** T_i and T_j of a graph G is also **defined** as $d(T_i, T_j) = [N(T_i \oplus T_j)]/2$, where $N(g)$ = the number of edges in a graph g.
  - $d(T_2, T_3) = 6/2 = 3$. Here, $N(T_2 \oplus T_3) = 6$, as $(T_2 \oplus T_3)$ represents the set of edges that are either in T_2 or T_3, but not in both i.e. {a, b, d, e, f, g}.
- The number $d(T_i, T_j)$ is the **minimum number of cyclic interchanges** involved in going from T_i to T_j.



A **Graph** and **three** of its **spanning trees**

Distance Between Two Spanning Trees

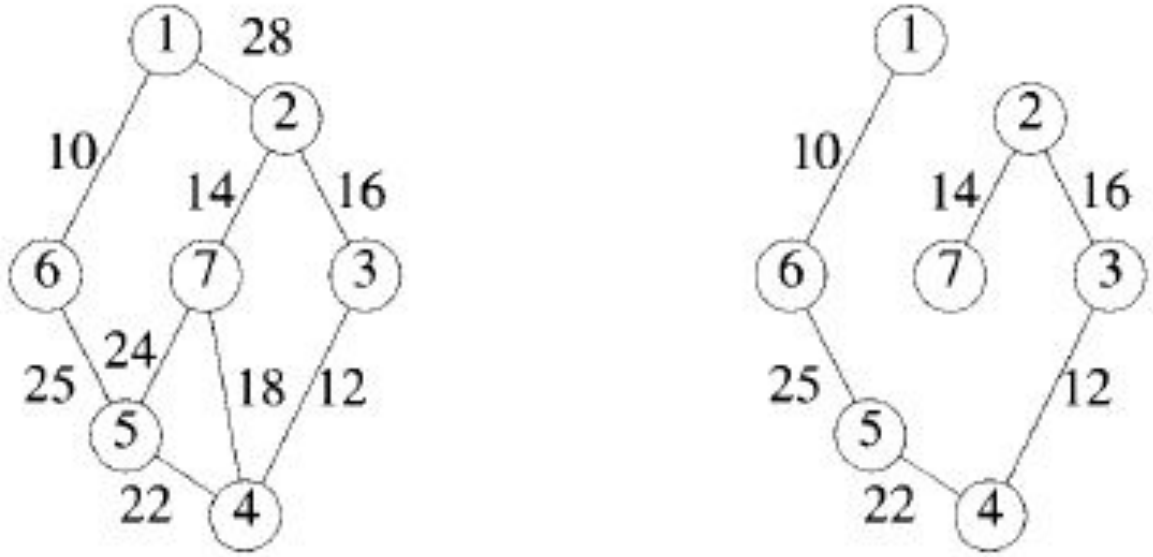
- The **distance** $d(T_i, T_j)$ **between two spanning trees** T_i and T_j of a graph **G** is a **metric**. That is, it satisfies
  - $d(T_i, T_j) \geq 0$ and $d(T_i, T_j) = 0$ if and only if $T_i = T_j$
  - $d(T_i, T_j) = d(T_j, T_i)$
  - $d(T_i, T_j) \leq d(T_i, T_k) + d(T_k, T_j)$
- To **prove** this we need to **show**
  - $d(T_i, T_j) \geq 0$ and $d(T_1, T_1) = 0$
  - $d(T_1, T_2) = d(T_2, T_1)$
  - $d(T_1, T_3) \leq d(T_1, T_2) + d(T_2, T_3)$



A **Graph** and **three** of its **spanning trees**

Spanning Trees in a Weighted Graph

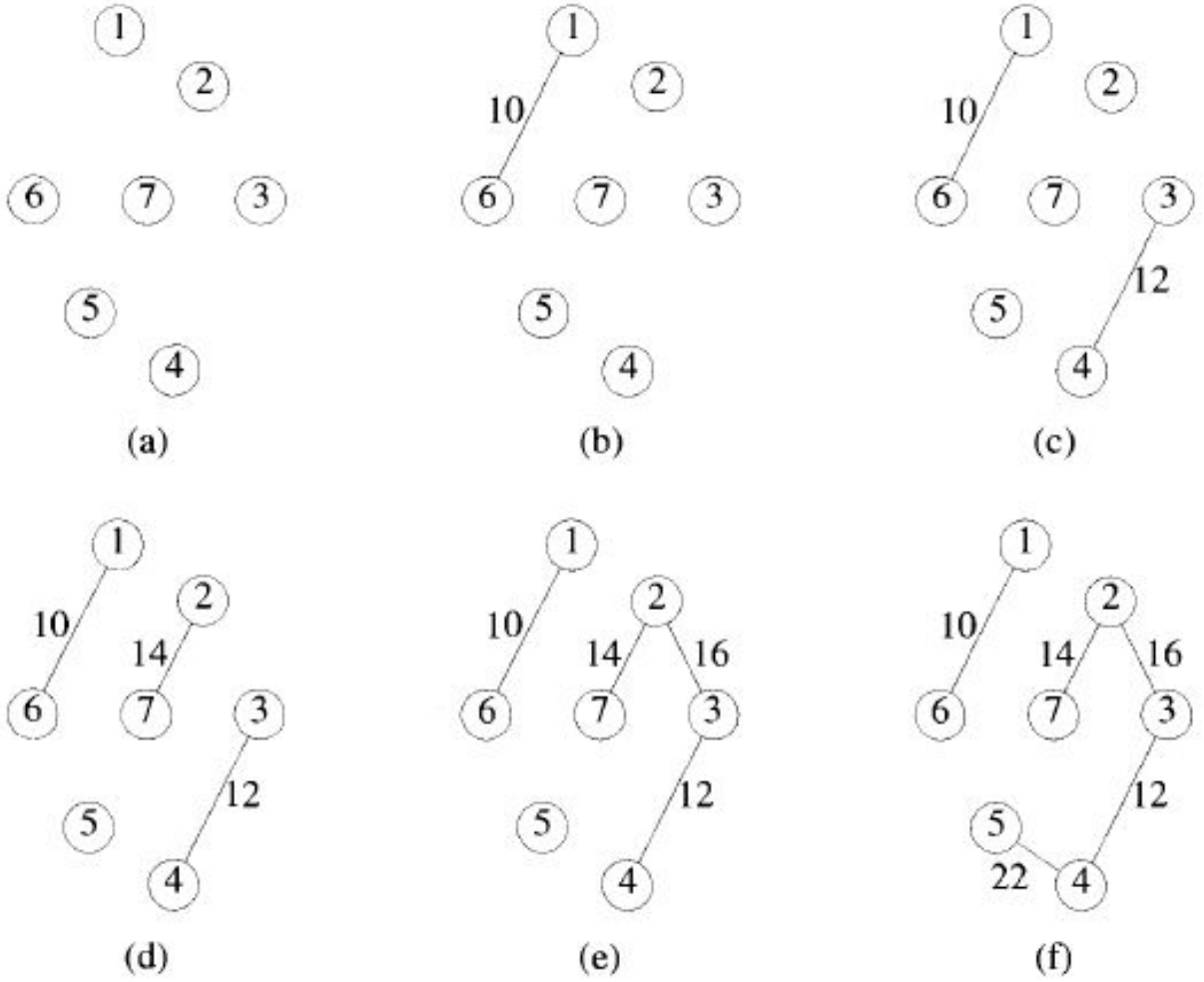
- In a weighted graph **G** (i.e., edges of **G** are associated **weights**), then the **weight** of a **spanning tree T** of **G** is **defined as the sum of the weights of all the branches** in **T**.
- Among **all the spanning trees** of **G**, **one** with the **smallest weight** in a weighted graph is called a **shortest spanning tree** or **shortest-distance spanning tree** or **minimal spanning tree**.
- **Application**: if we are to connect **n** cities $v_1, v_2, \ldots, v_n$ through a network of roads, then the solution is to find the least expensive network that connects all n cities together. This is equivalent to finding a shortest spanning tree in the connected weighted graph of n vertices.



A graph **G** and its **minimum spanning tree T**

Algorithm for Shortest Spanning Tree

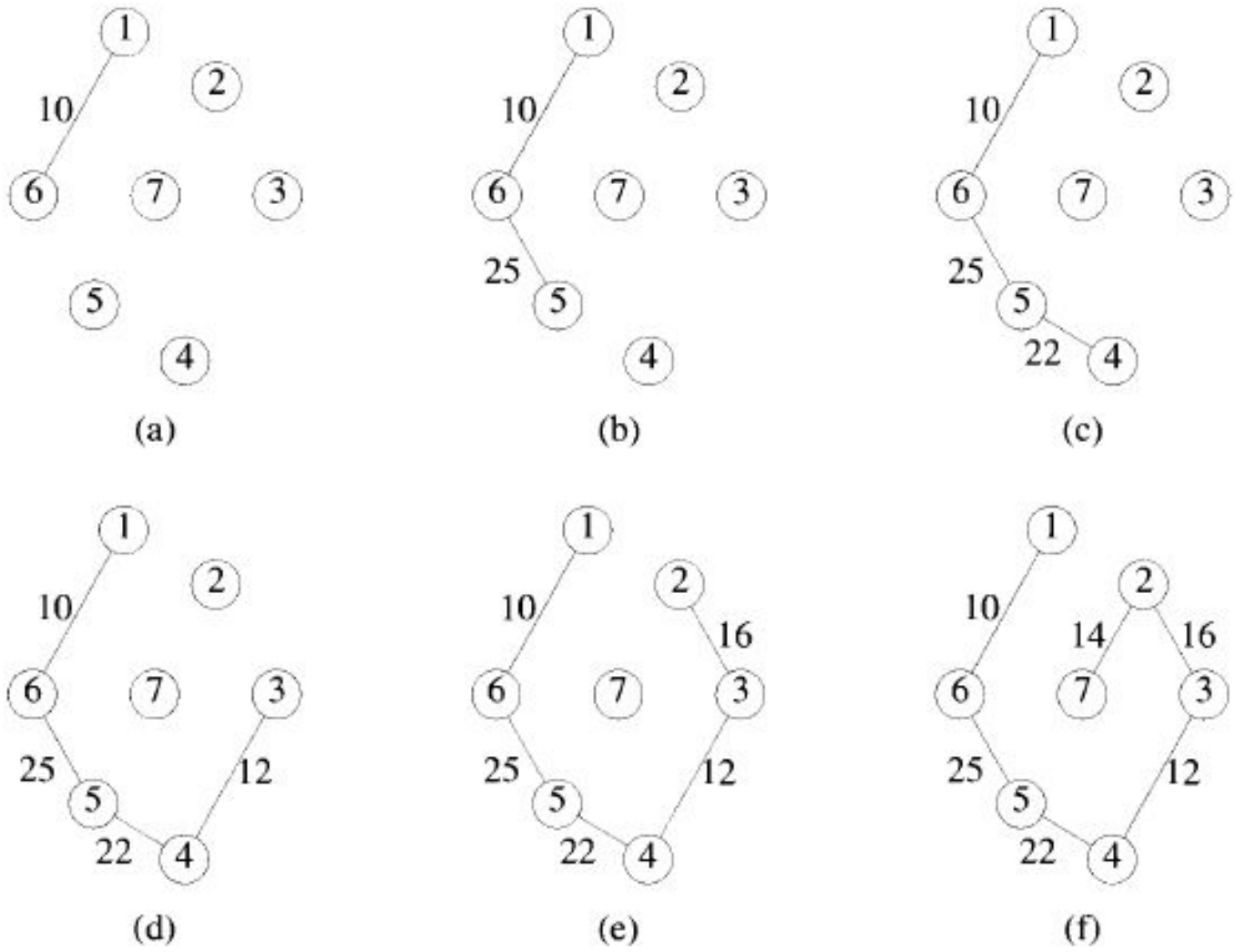
- **Kruskal algorithm**: **Prepare** the **list** **all edges** of the graph *G* **in order of** **non-decreasing** **weight**. Keep selecting edges from the list to be included in the spanning tree, that makes no circuit with the previously selected edges. **Continue** until ***n* – 1** edges have been selected, and **these edges will constitute the desired shortest spanning tree**.



√	√	√	√	X	√	X	√	X
10(1,6)	12(3,4)	14(2,7)	16(2,3)	18(4,7)	22(4,5)	24(5,7)	25(5,6)	28(1,2)

Algorithm for Shortest Spanning Tree

- **Prim's algorithm**: **Select edges one at a time** in to the **Spanning Tree to be created** in such a way that, at any stage the **set** of **edges A selected so far** is a **Tree**(which is not the case in Kruskal Algo).
- The algo. **starts** with a **tree** that **consists of only a minimum-cost edge of G**.
- Then **edges** are **added** to **this tree one by one**. The **next edge** (**i, j**) to be added is such that **i** is a **vertex already included** in the tree, **j** is a vertex **not yet included, and w**(**i, j**) is **minimum among all edges** (**k, l**) **such that vertex k** is **in the tree** and **vertex l** is **not in the tree**.



Stages of Prim's algorithm